

SOFTWARE DESIGN DOCUMENT (SDD)

Atlus — Sistema de Gestão de Atleticas Universitárias

Projeto: Atlus

Versão: 2.1

Data: 2026-08-22

Autores: Adama Augusto, Serio Pinheiro, Gabriel Esteves, André Arrais, Melissa Oliveira, Tainá Lima e Milton Junior

Status: Design Finalizado

ÍNDICE

1. [Visão Geral](#)
 2. [Requisitos do Sistema](#)
 3. [Arquitetura](#)
 4. [Banco de Dados](#)
 5. [Backend](#)
 6. [Frontend](#)
 7. [API REST](#)
 8. [Segurança](#)
 9. [Fluxos de Dados](#)
 10. [Performance e Escalabilidade](#)
 11. [Tratamento de Erros](#)
 12. [Deployment](#)
 13. [Operação e Manutenção](#)
-

1. VISÃO GERAL

1.1 O Que é Atlus

Atlus é uma plataforma SaaS de gestão para atleticas (organizações estudantis) em universidades federais brasileiras. O sistema permite que múltiplas atleticas gerenciem seus membros, products, eventos e financiamento de forma integrada, com suporte a desconto automático para sócios e roteamento automático baseado no curso do aluno.

1.2 Principais Funcionalidades

Gestão de Atleticas (Multi-Tenancy)

- Cada atletica é um "tenant" isolado no sistema
- Dados de uma atletica não são visíveis a outras
- Admin de cada atletica gerencia seus próprios usuários, produtos, eventos

RBAC (Role-Based Access Control)

- **DIRETORIA**: Acesso administrativo completo (CRUD de tudo, analytics)
- **SOCIO**: Acesso limitado (compra com desconto, inscrição em eventos com desconto)
- **ALUNO**: Acesso limitado (compra sem desconto, inscrição em eventos sem desconto)

Desconto Automático

- Sócios recebem desconto percentual automático em produtos e eventos
- Desconto é definido por atletica
- Visualização clara do desconto no checkout (preço original riscado, preço final em verde)

Autenticação Automática (RGA-Based)

- Alunos fazem login com RGA (Registro Geral Acadêmico) + senha
- Sistema automaticamente extrai código do curso do RGA
- Aluno é direcionado para sua atletica correta
- Se aluno trocou de curso, pode adicionar nova atletica pelo código do curso

Multi-Atletica

- Aluno pode ter acesso a múltiplas atleticas (se trocou de curso)
- Seletor no navbar permite trocar entre atleticas
- Perfil do usuário permite adicionar/remover atleticas

E-Commerce

- Produtos com estoque
- Carrinho e checkout
- Histórico de compras

Gestão de Eventos

- Criação de eventos (DIRETORIA)

- Inscrição em eventos (SOCIO/ALUNO)
- Histórico de participações
- Desconto automático para sócios

Analytics

- Dashboard para DIRETORIA com sales, registrations, insights
 - Dashboard pessoal para SOCIO/ALUNO
-

2. REQUISITOS DO SISTEMA

2.1 Funcionais (User Stories)

Autenticação e Autorização

- ☐ Login via RGA + senha
- ☐ Auto-discovery de atletica baseado no RGA
- ☐ Múltiplas atleticas por usuário
- ☐ RBAC com 3 roles (DIRETORIA, SOCIO, ALUNO)
- ☐ Permissões específicas por role
- ☐ Trocar atletica ativa (seletor)
- ☐ Adicionar/remover atletica no perfil

Gestão de Diretorias (Admin)

- ☐ CRUD de diretorias
- ☐ Código do curso por diretoria
- ☐ Desconto configurável por diretoria
- ☐ Status ativo/inativo

Gestão de Sócios

- ☐ CRUD de sócios (DIRETORIA only)
- ☐ Lista de sócios por atletica
- ☐ Associação user ↔ socio

E-Commerce (Loja)

- ☐ CRUD de produtos (DIRETORIA)
- ☐ Grid de produtos (SOCIO/ALUNO)

- ☐ Carrinho (sessão)
- ☐ Checkout com desconto automático
- ☐ Histórico de compras
- ☐ Analytics de vendas (DIRETORIA)

Gestão de Eventos

- ☐ CRUD de eventos (DIRETORIA)
- ☐ Lista de eventos (SOCIO/ALUNO)
- ☐ Inscrição em eventos
- ☐ Desconto automático para sócios
- ☐ Histórico de participações
- ☐ Analytics de registrations (DIRETORIA)

Perfil de Usuário

- ☐ Dados pessoais
- ☐ Seção "Atletica" (multi-atletica management)
- ☐ Histórico de compras
- ☐ Histórico de eventos

Dashboard

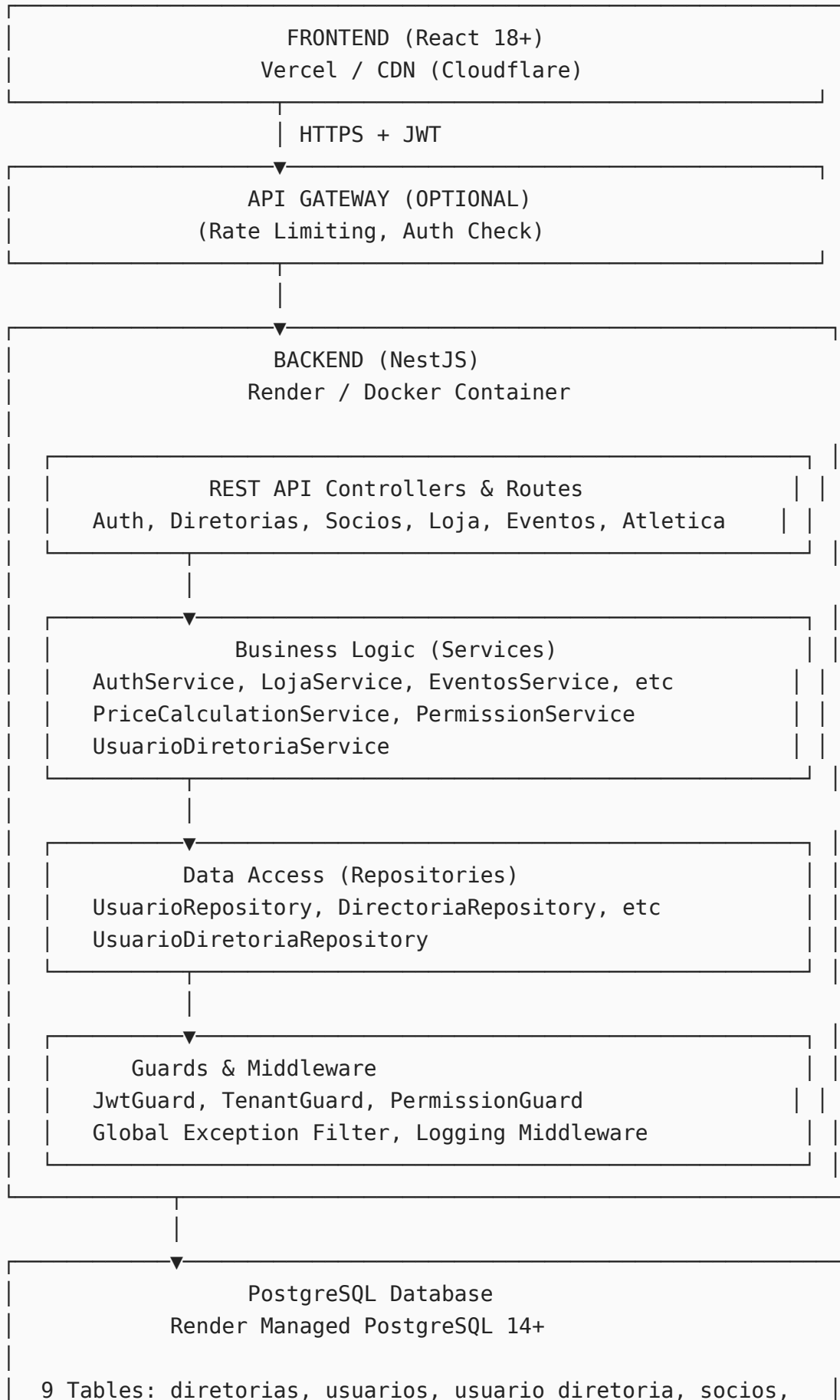
- ☐ Dashboard para DIRETORIA (analytics completo)
- ☐ Dashboard pessoal para SOCIO/ALUNO

2.2 Não-Funcionais

- **Performance:** Response time < 200ms (p95)
- **Disponibilidade:** Uptime 99.5%
- **Escalabilidade:** Suportar 1000+ req/s
- **Segurança:** JWT, bcrypt, SQL injection protection, tenant isolation
- **Escalabilidade Horizontal:** Backend stateless (múltiplas instâncias)
- **Caching:** Redis para sessões e desconto cache
- **CDN:** Imagens via CDN (Cloudflare)

3. ARQUITETURA

3.1 Visão Geral



produtos, vendas, eventos, participacoes, loja

3.2 Padrões Arquiteturais

Layered Architecture

```
Presentation Layer
    ↓
API/Controller Layer
    ↓
Business Logic Layer
    ↓
Data Access Layer (Repository)
    ↓
Database Layer
```

Multi-Tenancy (Database-Level Isolation)

Todas as queries incluem: WHERE diretoria_id = ?
Garantindo isolamento entre atleticas

Dependency Injection (NestJS Native)

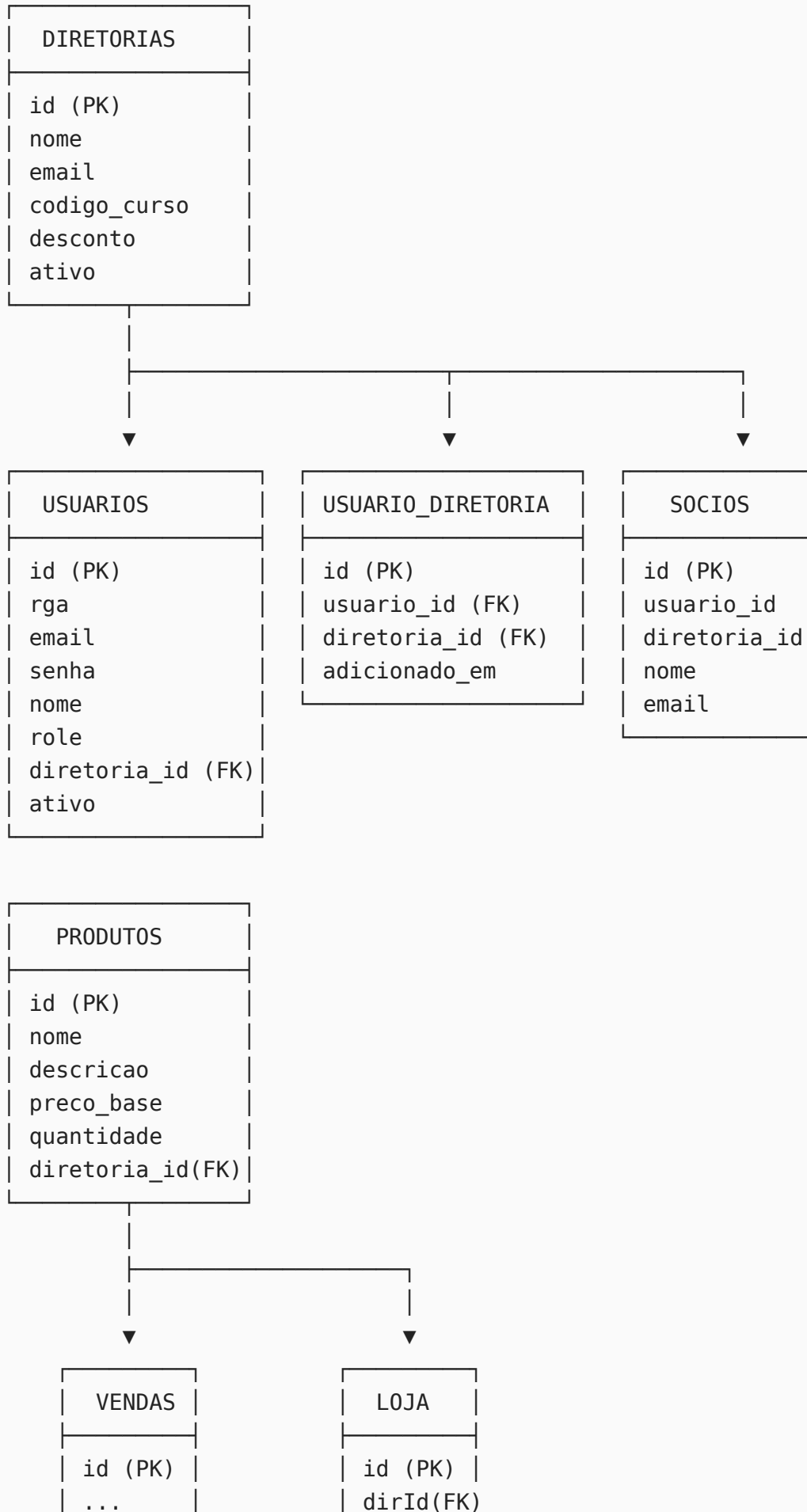
Services injetados em Controllers
Repositories injetados em Services
Fácil para testes e manutenção

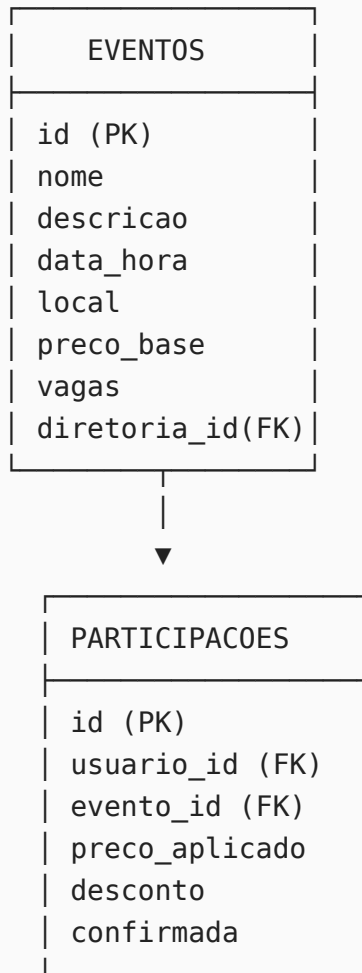
Guard Pattern (Middleware de Segurança)

JwtGuard → valida token
TenantGuard → valida diretoria
PermissionGuard → valida permissões

4. BANCO DE DADOS

4.1 Modelo Conceitual





4.2 Tabelas (Prisma Schema)

```
// Diretoria (Tenant)
model Diretoria {
  id          String    @id @default(cuid())
  nome        String
  email       String    @unique
  codigoCurso String    @unique           // Identifica o curso
  desconto    Float     @default(0.0)    // Desconto padrão (%)
  ativo       Boolean   @default(true)
  criadoEm    DateTime  @default(now())
  atualizadoEm DateTime  @updatedAt

  // Relações
  usuarios          Usuario[] @relation("principal")
  usuariosDiretorias UsuarioDiretoria[]
  socios            Socio[]
  produtos          Produto[]
}
```

```

    eventos          Evento[]
    loja              Loja?

    @@unique([codigoCurso])
}

// Usuario
model Usuario {
    id                String    @id @default(cuid())
    rga                String    @unique
    email             String    @unique
    senha             String
    nome              String
    role              Role      @default(ALUNO)

    // Diretoria principal (auto-atribuída pelo RGA)
    diretoriaId       String
    diretoria          Diretoria @relation("principal",
        fields: [diretoriaId],
        references: [id],
        onDelete: Cascade)

    // Multi-atletica
    directorias        UsuarioDiretoria[]

    ativo              Boolean   @default(true)
    criadoEm           DateTime  @default(now())
    atualizadoEm       DateTime  @updatedAt

    // Relações
    socios             Socio[]
    vendas             Venda[]
    participacoes      Participacao[]

    @@index([diretoriaId])
    @@index([rga])
}

// UsuarioDiretoria (Multi-Atletica)
model UsuarioDiretoria {
    id                String    @id @default(cuid())

    usuarioId         String
    usuario            Usuario  @relation(fields: [usuarioId],
        references: [id], onDelete: Cascade)

```

```

    diretoriaId String
    diretoria    Diretoria @relation(fields: [diretoriaId],
                                     references: [id], onDelete: Cascade)

    adicionadoEm DateTime @default(now())

    @@unique([usuarioId, diretoriaId])
    @@index([usuarioId])
    @@index([diretoriaId])
}

// Socio
model Socio {
    id          String    @id @default(cuid())
    usuarioId   String
    usuario     Usuario   @relation(fields: [usuarioId],
                                     references: [id])
    diretoriaId String
    diretoria    Diretoria @relation(fields: [diretoriaId],
                                     references: [id], onDelete: Cascade)
    criadoEm     DateTime @default(now())

    @@index([diretoriaId])
    @@unique([usuarioId, diretoriaId])
}

// Produto
model Produto {
    id          String    @id @default(cuid())
    nome        String
    descricao   String?
    precoBase   Float
    quantidade  Int        @default(0)
    diretoriaId String
    diretoria    Diretoria @relation(fields: [diretoriaId],
                                     references: [id], onDelete: Cascade)
    criadoEm     DateTime @default(now())
    atualizadoEm DateTime @updatedAt

    vendas       Venda[]

    @@index([diretoriaId])
}

// Venda
model Venda {

```

```

    id          String    @id @default(cuid())
    usuarioId   String
    usuario     Usuario   @relation(fields: [usuarioId],
                                references: [id])
    produtoNome String
    produtoPreco Float    // Preço aplicado (com desconto)
    desconto    Float     // Desconto (%)
    quantidade  Int
    total       Float     // total = produtoPreco * quantidade
    criadoEm    DateTime  @default(now())

```

```

    @@index([usuarioId])
}

```

// Evento

```

model Evento {
    id          String    @id @default(cuid())
    nome        String
    descricao   String?
    dataHora    DateTime
    local       String
    precoBase   Float     @default(0.0)
    vagas       Int?
    diretoriaId String
    diretoria   Diretoria @relation(fields: [diretoriaId],
                                references: [id], onDelete: Cascade)
    criadoEm    DateTime  @default(now())
    atualizadoEm DateTime  @updatedAt

```

```

    participacoes Participacao[]

```

```

    @@index([diretoriaId])
    @@index([dataHora])
}

```

// Participacao

```

model Participacao {
    id          String    @id @default(cuid())
    usuarioId   String
    usuario     Usuario   @relation(fields: [usuarioId],
                                references: [id], onDelete: Cascade)
    eventoId    String
    evento      Evento    @relation(fields: [eventoId],
                                references: [id], onDelete: Cascade)
    precoAplicado Float?
    desconto    Float     @default(0.0)

```

```

    confirmada      Boolean @default(false)
    pagamentoPendente Boolean @default(true)
    criadoEm        DateTime @default(now())

    @@unique([usuarioId, eventoId])
    @@index([usuarioId])
    @@index([eventoId])
}

// Loja
model Loja {
    id          String  @id @default(cuid())
    diretoriaId String  @unique
    diretoria   Diretoria @relation(fields: [diretoriaId],
                                     references: [id], onDelete: Cascade)
    criadoEm    DateTime @default(now())
}

enum Role {
    DIRETORIA
    SOCIO
    ALUNO
}

```

4.3 Índices para Performance

```

-- Multi-tenant queries (mais comuns)
CREATE INDEX idx_usuarios_diretoria_role
  ON usuarios(diretoria_id, role);
CREATE INDEX idx_socios_diretoria
  ON socios(diretoria_id);
CREATE INDEX idx_produtos_diretoria
  ON produtos(diretoria_id);
CREATE INDEX idx_eventos_diretoria_data
  ON eventos(diretoria_id, data_hora);

-- Multi-atletica
CREATE INDEX idx_usuario_diretoria_usuario
  ON usuario_diretoria(usuario_id);
CREATE INDEX idx_usuario_diretoria_diretoria
  ON usuario_diretoria(diretoria_id);

-- Lookup por RGA (login)
CREATE INDEX idx_usuarios_rga
  ON usuarios(rga);

```

```
-- Lookup por código de curso (auto-routing)
CREATE INDEX idx_diretorias_codigo_curso
  ON diretorias(codigo_curso);
```

5. BACKEND

5.1 Stack

- **Framework:** NestJS 9+
- **Linguagem:** TypeScript 4.9+
- **Runtime:** Node.js 18+
- **ORM:** Prisma 4+
- **Database:** PostgreSQL 14+
- **Auth:** JWT (jsonwebtoken)
- **Password:** bcrypt
- **Validation:** class-validator, class-transformer

5.2 Arquitetura de Módulos

```
src/
├── app.module.ts
├── main.ts
├──
├── auth/
│   ├── auth.module.ts
│   ├── auth.controller.ts
│   ├── auth.service.ts
│   ├── rga-parser.service.ts
│   ├── jwt.strategy.ts
│   ├── jwt.guard.ts
│   ├── dtos/
│   │   ├── login.dto.ts
│   │   └── auth-response.dto.ts
│   └── types/
│       └── token-payload.ts
├──
├── diretorias/
│   ├── diretorias.module.ts
│   ├── diretorias.controller.ts
│   └── diretorias.service.ts
```

```
|   ├── diretorias.repository.ts
|   ├── dtos/
|   |   ├── create-diretoria.dto.ts
|   |   └── update-diretoria.dto.ts
|
|── usuarios/
|   ├── usuarios.module.ts
|   ├── usuarios.controller.ts
|   ├── usuarios.service.ts
|   ├── usuarios.repository.ts
|   └── dtos/
|
|── usuario-diretoria/ (Multi-Atletica)
|   ├── usuario-diretoria.module.ts
|   ├── atletica.controller.ts
|   ├── usuario-diretoria.service.ts
|   ├── usuario-diretoria.repository.ts
|   ├── dtos/
|   |   └── add-atletica.dto.ts
|
|── socios/
|   ├── socios.module.ts
|   ├── socios.controller.ts
|   ├── socios.service.ts
|   ├── socios.repository.ts
|   └── dtos/
|
|── loja/
|   ├── loja.module.ts
|   ├── loja.controller.ts
|   ├── loja.service.ts
|   ├── repositories/
|   |   ├── loja.repository.ts
|   |   └── vendas.repository.ts
|   ├── dtos/
|   |   ├── compra.dto.ts
|   |   └── venda-response.dto.ts
|
|── eventos/
|   ├── eventos.module.ts
|   ├── eventos.controller.ts
|   ├── eventos.service.ts
|   ├── repositories/
|   |   ├── eventos.repository.ts
|   |   └── participacoes.repository.ts
|   └── dtos/
```

```

|   |   |─ create-evento.dto.ts
|   |   └─ inscricao.dto.ts
|
|─ shared/
|   |─ shared.module.ts
|   |─ guards/
|   |   |─ jwt.guard.ts
|   |   |─ tenant.guard.ts
|   |   └─ permission.guard.ts
|   |─ decorators/
|   |   |─ current-user.decorator.ts
|   |   |─ current-tenant.decorator.ts
|   |   └─ require-permission.decorator.ts
|   |─ filters/
|   |   └─ http-exception.filter.ts
|   |─ middleware/
|   |   └─ logging.middleware.ts
|   |─ services/
|   |   |─ permission.service.ts
|   |   └─ price-calculation.service.ts
|   |─ types/
|   |   └─ token-payload.ts
|   └─ constants/
|       |─ roles.ts
|       └─ permissions.ts
└─ database/
    |─ prisma.service.ts
    └─ migrations/

```

5.3 Services Principais

AuthService

```

login(rga, senha): Promise<AuthResponse>
  1. Validar credenciais (SIA ou local)
  2. Extrair código do RGA
  3. Buscar diretoria principal
  4. Buscar diretorias extras (usuario_diretoria)
  5. Gerar JWT com todas

validateToken(token): Promise<TokenPayload>
  1. Validar assinatura
  2. Retornar payload decodificado

```

UsuarioDiretoriaService

`addDiretoria(usuarioId, codigoCurso): Promise<UsuarioDiretoria>`

1. Buscar diretoria por código
2. Validar se já tem acesso
3. Criar relação usuario_diretoria

`removeDiretoria(usuarioId, diretoriaId): Promise<void>`

1. Validar se não é diretoria principal
2. Deletar relação

`getDiretorias(usuarioId): Promise<Diretoria[]>`

1. Buscar todas as diretorias do usuário

LojaService

`getProdutos(diretoriaId): Promise<Produto[]>`

1. Listar produtos da diretoria

`comprar(compraDto, user): Promise<Venda>`

1. Validar produto existe
2. Calcular preço com desconto (PriceCalculationService)
3. Criar Venda
4. Decrementar estoque

`getVendas(diretoriaId): Promise<Venda[]>`

1. Analytics: listar vendas
2. Filtrar por diretoria

EventosService

`getEventos(diretoriaId): Promise<Evento[]>`

1. Listar eventos da diretoria

`inscrever(eventoId, user): Promise<Participacao>`

1. Validar evento existe
2. Validar vagas disponíveis
3. Calcular preço com desconto
4. Criar Participacao

`getParticipacoes(eventoId): Promise<Participacao[]>`

1. Analytics: listar participações

PriceCalculationService

`calculateProductPrice(basePrice, role, desconto): PriceResult`

1. If role == **SOCIO**: aplicar desconto
2. Else: sem desconto
3. Return { precoFinal, desconto, economia }

`calculateEventPrice(basePrice, role, desconto): PriceResult`

1. If role == **SOCIO**: aplicar desconto
2. Else: sem desconto
3. Return { precoFinal, desconto, economia }

PermissionService

`getPermissionsByRole(role): string[]`

1. **DIRETORIA**: retorna todas as permissões
2. **SOCIO**: retorna permissões limitadas
3. **ALUNO**: retorna permissões limitadas

`canAccess(user, permission): boolean`

1. Verificar se user.permissions inclui permission

5.4 Guards

JwtGuard

`canActivate(context):`

1. Extrair **JWT** do Authorization header
2. Validar assinatura
3. Decodificar payload
4. Injeta em request.user
5. Se inválido: **401** Unauthorized

TenantGuard

`canActivate(context):`

1. Validar user.diretoriaAtiva existe
2. Validar diretoriaAtiva está em user.directorias
3. Injeta request.diretoriaId e request.directorias
4. Se inválido: **403** Forbidden

PermissionGuard

```
canActivate(context):  
  1. Ler metadados @RequirePermission()  
  2. Validar user.permissions inclui permission  
  3. Se inválido: 403 Forbidden
```

6. FRONTEND

6.1 Stack

- **Framework:** React 18+
- **Build:** Vite
- **Styling:** Tailwind CSS
- **HTTP:** Axios
- **State:** React Query + Context API
- **Forms:** React Hook Form + Zod
- **Icons:** Lucide React
- **Components:** shadcn/ui (opcional)

6.2 Estrutura de Diretórios

```
src/  
├─ App.tsx  
├─ main.tsx  
├─ index.css  
├─  
├─ components/  
│   └─ Common/  
│       ├── Button.tsx  
│       ├── Card.tsx  
│       ├── Modal.tsx  
│       ├── Input.tsx  
│       ├── Select.tsx  
│       ├── Alert.tsx  
│       ├── Spinner.tsx  
│       ├── Badge.tsx  
│       ├── Skeleton.tsx  
│       └─ Table.tsx  
├─  
├─ Layout/  
│   └─ Navbar.tsx
```

```
| | | └─ Sidebar.tsx
| | | └─ Footer.tsx
| | | └─ ProtectedRoute.tsx
| |
| └─ Modules/
| | | └─ AteticaDropdown.tsx      (Multi-Atletica)
| | | └─ AteticaSelector.tsx     (Profile)
| | | └─ ProductCard.tsx
| | | └─ EventCard.tsx
| | | └─ SociosTable.tsx
| | | └─ ProdutosTable.tsx
| | | └─ EventosTable.tsx
| |
| └─ ErrorBoundary.tsx
|
└─ pages/
  | └─ LoginPage.tsx
  | └─ DashboardPage.tsx
  | └─ ProfilePage.tsx
  |   └─ seções: Personal, Atletica, Security
  | └─ SociosPage.tsx
  | └─ ProdutosPage.tsx
  | └─ LojaPage.tsx
  | └─ MinhasComprasPage.tsx
  | └─ EventosPage.tsx
  | └─ MeusEventosPage.tsx
  | └─ ErrorPage.tsx
  |
  └─ context/
    | └─ AuthContext.tsx
    |   | └─ user
    |   | └─ token
    |   | └─ diretoriaAtiva
    |   | └─ directorias
    |   | └─ login()
    |   | └─ logout()
    |   | └─ switchAtletica()
    |   └─ ThemeContext.tsx
    |
    └─ hooks/
      | └─ useAuth.ts
      | └─ useForm.ts
      | └─ useFetch.ts
      | └─ useQuery.ts (React Query)
      |
      └─ services/
```

```

|   ├── api.ts           (Axios instance com interceptors)
|   ├── auth.ts          (POST /auth/login)
|   ├── diretorias.ts
|   ├── socios.ts
|   ├── loja.ts
|   ├── eventos.ts
|   └── atletica.ts      (GET/POST/DELETE /atletica)
|
|   ├── types/
|   |   ├── User.ts
|   |   ├── Diretoria.ts
|   |   ├── Produto.ts
|   |   ├── Evento.ts
|   |   └── ApiResponse.ts
|   |
|   ├── utils/
|   |   ├── auth.ts      (JWT parsing, validation)
|   |   ├── format.ts    (Formatação de valores)
|   |   └── validation.ts (Schemas Zod)
|   |
|   └── styles/
|       ├── globals.css
|       ├── components.css
|       └── tailwind.config.ts

```

6.3 Principais Componentes

Navbar

```

| Logo
| AteticaDropdown (se diretorias.length > 1)
|   ├── Button: "🏛️ Xadrez ▼"
|   └── Menu:
|       ├── Xadrez (Ativa) ✓
|       ├── Dança
|       └── Esportes
|   UserDropdown
|       ├── Profile
|       ├── Settings
|       └── Logout
|   ThemeToggle

```

ProfilePage - Atletica Section

- └─ Atletica **Principal** (read-only)
- └─ Minhas **Atleticas** (lista com Remove buttons)
 - └─ Adicionar Outra Atletica
 - └─ Input: **codigo_curso**
 - └─ Helper text
 - └─ Button: **Adicionar**

ProductCard

- └─ Imagem
- └─ Nome
- └─ Preço **Original** (riscado, se desconto)
- └─ Desconto % (verde, se aplicável)
- └─ Preço Final
- └─ Button: **Comprar**

EventCard

- └─ Imagem/Banner
- └─ Data e hora
- └─ Local
- └─ Preço **Original** (riscado, se desconto)
- └─ Desconto % (verde)
- └─ Preço Final
- └─ Vagas disponíveis
- └─ Button: **Inscrever**

6.4 State Management

AuthContext

```
user: {  
  id: string  
  rga: string  
  nome: string  
  role: Role  
  diretoriaAtiva: Diretoria  
  directorias: Diretoria[]  
}  
token: string  
diretoriaAtiva: Diretoria  
directorias: Diretoria[]
```

```
login(rga, senha): Promise<void>
logout(): void
switchAtletica(diretoriaId): Promise<void>
```

React Query

```
// Queries
const { data: produtos } = useQuery('produtos', getProdutos)
const { data: eventos } = useQuery('eventos', getEventos)

// Mutations
const comprarMutation = useMutation(comprar)
const inscreverMutation = useMutation(inscrever)
```

7. API REST

7.1 Endpoints de Autenticação

```
POST /auth/login
  Request: { rga, senha }
  Response: { access_token, user: { id, nome, diretoriaAtiva, directorias } }
  Status: 200 | 401

POST /auth/logout
  Response: { message: "Logout realizado" }
  Status: 200
```

7.2 Endpoints de Atletica (Multi-Atletica)

```
GET /atletica
  Headers: Authorization: Bearer <token>
  Response: Diretoria[]
  Status: 200

POST /atletica
  Headers: Authorization: Bearer <token>
  Body: { codigoCurso: string }
  Response: { message, diretoria: Diretoria }
  Status: 201 | 409 | 404
```

```
DELETE /atletica/:diretoriaId
  Headers: Authorization: Bearer <token>
  Response: { message: "Atletica removida" }
  Status: 200 | 400 | 404

PUT /atletica/ativa/:diretoriaId
  Headers: Authorization: Bearer <token>
  Response: { message, novaAteticaAtiva: string }
  Status: 200 | 403 | 404
```

7.3 Endpoints de Diretorias (Admin)

```
GET /diretorias
  Headers: Authorization: Bearer <token>
  Response: Diretoria[]
  Status: 200

POST /diretorias
  Headers: Authorization: Bearer <token>
  Body: { nome, email, codigoCurso, desconto }
  Response: Diretoria
  Status: 201 | 409 (duplicate codigo_curso)

PATCH /diretorias/:id
  Headers: Authorization: Bearer <token>
  Body: { nome?, email?, desconto?, ativo? }
  Response: Diretoria
  Status: 200 | 404

DELETE /diretorias/:id
  Headers: Authorization: Bearer <token>
  Response: { message: "Diretoria deletada" }
  Status: 200 | 404 | 409 (usuários associados)
```

7.4 Endpoints de Socios

```
GET /socios
  Query: ?diretoriaId=...
  Response: Socio[]
  Status: 200

POST /socios
  Body: { usuarioId, nome, email }
  Response: Socio
```

Status: 201

PATCH /socios/:id

Body: { nome?, email? }

Response: Socio

Status: 200 | 404

DELETE /socios/:id

Response: { message: "Socio removido" }

Status: 200 | 404

7.5 Endpoints de Loja

GET /loja/produtos

Response: Produto[]

Status: 200

POST /loja/comprar

Headers: Authorization: Bearer <token>

Body: { produtoId, quantidade }

Response: { sucesso, venda, precoFinal, desconto, mensagem }

Status: 200 | 400 | 404

GET /loja/minhas-compras

Headers: Authorization: Bearer <token>

Response: Venda[]

Status: 200

GET /loja/vendas

Headers: Authorization: Bearer <token> (DIRETORIA only)

Response: { total, vendas: Venda[], analyticsData }

Status: 200

7.6 Endpoints de Eventos

GET /eventos

Response: Evento[]

Status: 200

POST /eventos

Headers: Authorization: Bearer <token> (DIRETORIA only)

Body: { nome, descricao, dataHora, local, precoBase, vagas }

Response: Evento

Status: 201 | 403

POST /eventos/:id/inscrever

Headers: Authorization: Bearer <token>

Response: { sucesso, participacao, precoFinal, desconto }

Status: 201 | 400 | 404

GET /eventos/:id/participacoes

Headers: Authorization: Bearer <token> (DIRETORIA only)

Response: Participacao[]

Status: 200

GET /meus-eventos

Headers: Authorization: Bearer <token>

Response: Participacao[]

Status: 200

8. SEGURANÇA

8.1 Autenticação

JWT (JSON Web Token)

Header:

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Payload:

```
{  
  "userId": "usr-123",  
  "rga": "20240001234",  
  "role": "SOCIO",  
  "diretoriaAtiva": "xadrez-123",  
  "desconto": 10.0,  
  "directorias": ["xadrez-123", "danca-456", "esportes-789"],  
  "permissions": ["dashboard:own", "loja:read", "loja:purchase", ...],  
  "iat": 1692806400,  
  "exp": 1692892800  
}
```

```
Signature: HMACSHA256(base64UrlEncode(header) + "." +  
base64UrlEncode(payload), secret)
```

Ciclo de Vida

- **Login:** Backend emite JWT (validade: 24h)
- **Request:** Frontend inclui JWT no header Authorization
- **Validação:** JwtGuard valida assinatura
- **Expiração:** Após 24h, user faz logout automático (no frontend)

8.2 Autorização

RBAC (Role-Based Access Control)

```
DIRETORIA (15 permissões):  
├─ dashboard:analytics  
├─ socios:create, socios:read, socios:update, socios:delete  
├─ produtos:create, produtos:read, produtos:update, produtos:delete  
├─ loja:manage, loja:analytics  
├─ eventos:create, eventos:read, eventos:update, eventos:delete  
└─ eventos:analytics  
  
SOCIO (5 permissões):  
├─ dashboard:own  
├─ loja:read, loja:purchase  
└─ eventos:read, eventos:participate  
  
ALUNO (4 permissões):  
├─ dashboard:own  
├─ loja:read, loja:purchase  
└─ eventos:read, eventos:participate
```

Decorador @RequirePermission()

```
@Post('socios')  
@RequirePermission('socios:create')  
async create(@Body() dto: CreateSocioDto) { ... }
```

8.3 Isolamento de Tenant (Multi-Tenancy)

Row-Level Security

```
// OBRIGATÓRIO: Sempre filtrar por diretoriaId

// ✅ CORRETO:
async getSocios(diretoriaId: string) {
  return this.prisma.socio.findMany({
    where: { diretoriaId }
  });
}

// ❌ ERRADO:
async getAllSocios() {
  return this.prisma.socio.findMany(); // Bug de segurança!
}
```

TenantGuard

```
// Validar se user tem acesso a diretoria ativa
if (!user.directorias.includes(user.diretoriaAtiva)) {
  throw new ForbiddenException();
}
```

8.4 Proteção contra Ataques Comuns

SQL Injection

✅ Protegido: Prisma parametriza queries automaticamente

CSRF (Cross-Site Request Forgery)

✅ JWT é stateless (não usa cookies) ✅ CORS configurado

XSS (Cross-Site Scripting)

✅ React sanitiza rendering por padrão ✅ Validação de input (Zod)

Brute Force

✅ Rate limiting em /auth/login

Password Storage

✅ Bcrypt com 10 rounds ✅ Nunca armazenar plain text

8.5 HTTPS

-  Forçado em production
 -  Certificado SSL/TLS
-

9. FLUXOS DE DADOS

9.1 Fluxo de Login com Auto-Atletica

1. User abre app → LoginPage
2. Insere: RGA + Senha
3. POST /auth/login

[Backend]

4. AuthService.login(rga, senha)
5. Valida credenciais (SIA ou bcrypt)
6. Extrai código do RGA: "20240001234" → "01"
7. SELECT * FROM diretorias WHERE codigo_curso = "01"
→ Encontra: Diretoria(id="xadrez-123", nome="Xadrez")
8. SELECT * FROM usuario_diretoria WHERE usuario_id = ?
→ Encontra: [danca-456, esportes-789]
9. Compila JWT com:
 - diretoriaAtiva: "xadrez-123"
 - directorias: ["xadrez-123", "danca-456", "esportes-789"]
10. Return: { access_token, user }

[Frontend]

11. AuthContext.login(): salva token e user
12. localStorage.setItem('token', access_token)
13. Navigate to /dashboard
14. Navbar mostra:  Xadrez" (diretoriaAtiva)

9.2 Fluxo de Trocar Atletica

[Frontend]

1. User clica em dropdown Navbar
2. Vê: Xadrez (Ativa), Dança, Esportes
3. Clica em "Dança"
4. Chama: switchAtletica("danca-456")
5. PUT /atletica/ativa/danca-456

[Backend]

6. AteticaController.switchAtletica()
7. Valida: user.directorias.includes("danca-456") ✓

```
8. UPDATE usuarios SET diretoria_id = "danca-456"
9. Return: { message, novaAteticaAtiva: "danca-456" }
```

[Frontend]

```
10. AuthContext atualiza: diretoriaAtiva = "danca-456"
11. Próximas requests incluem: Authorization: Bearer <token>
    (Token antigo ainda válido, TenantGuard usa diretoriaAtiva do header ou context)
12. Dashboard/Loja/Eventos filtram por "danca-456"
13. Navbar mostra: "🏛️ Dança" (novo)
```

9.3 Fluxo de Compra com Desconto

[Frontend]

```
1. User vê ProductCard
  - Preço: R$ 50
  - User.role = "SOCIO", User.desconto = 10%
  - Exibe:
    * R$ 50 (riscado, vermelho)
    * -R$ 5 (em verde)
    * R$ 45 (preço final, grande)
2. Clica "Comprar"
3. CheckoutModal abre
4. Clica "Confirmar Compra"
5. POST /loja/comprar
```

[Backend]

```
6. LojaService.comprar(compraDto, user)
7. Busca Produto(id) → precoBase = 50
8. PriceCalculationService.calculateProductPrice()
  - basePrice = 50
  - role = "SOCIO"
  - desconto = 10
  - return { precoFinal: 45, desconto: 10, economia: 5 }
9. Cria Venda(usuarioId, produtoNome, produtoPreco=45, desconto=10, total=45)
10. UPDATE produtos SET quantidade = quantidade - 1
11. Return: { sucesso: true, precoFinal: 45, desconto: 10 }
```

[Frontend]

```
12. Toast: "Compra realizada! Economizou R$ 5"
13. Refetch: GET /loja/minhas-compras
14. Mostra nova compra no histórico
```

10. PERFORMANCE E ESCALABILIDADE

10.1 Backend

Caching

- Redis para session cache
- Cache de diretorias por código_curso
- Cache de permissões por role

Paginação

```
GET /loja/produtos?page=1&limit=10
```

Database Optimization

- Índices em (diretoriald, field)
- Índices em campos de lookup frequente
- EXPLAIN ANALYZE para queries lentas

10.2 Frontend

Code Splitting

```
const DashboardPage = React.lazy(() => import('./pages/DashboardPage'))  
const LojaPage = React.lazy(() => import('./pages/LojaPage'))
```

Image Optimization

- Lazy loading
- srcSet para diferentes resoluções
- WebP format

Bundle Optimization

- Tree shaking
- Minification
- Vite plugin-visualizer para análise

10.3 Infraestrutura

Horizontal Scaling

- Backend stateless (múltiplas instâncias Render)
- Database: PostgreSQL Managed
- Frontend: CDN Vercel

Load Balancing

- Automático via Render/Vercel
-

11. TRATAMENTO DE ERROS

11.1 Global Exception Filter

```
@Catch()  
export class AllExceptionsFilter implements ExceptionFilter {  
  catch(exception: unknown, host: ArgumentsHost) {  
    // Retorna: { statusCode, message, timestamp }  
  }  
}
```

11.2 Erros HTTP Padrão

- 400 Bad Request
 - Validação falhou
 - Campo obrigatório faltando
 - Formato inválido
- 401 Unauthorized
 - JWT inválido
 - Credenciais incorretas
- 403 Forbidden
 - Permissão insuficiente
 - Acesso negado a tenant
- 404 Not Found
 - Recurso não encontrado
- 409 Conflict
 - Recurso duplicado
 - Estado inválido

```
500 Internal Server Error
  - Erro não tratado
```

11.3 Logging

```
logger.log('User login', 'AuthService')
logger.warn('Low stock', 'LojaService')
logger.error('Database connection failed', error, 'PrismaService')
```

12. DEPLOYMENT

12.1 Backend (Render)

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
RUN npm run build
EXPOSE 3000
CMD ["npm", "run", "start:prod"]
```

Environment Variables

```
DATABASE_URL=postgresql://...
JWT_SECRET=...
JWT_EXPIRATION=24h
NODE_ENV=production
LOG_LEVEL=info
```

CI/CD (GitHub Actions)

```
on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
```

```
- uses: actions/checkout@v3
- name: Build and deploy
  run: |
    docker build -t atlus-backend .
    docker push ...
- name: Deploy to Render
  run: curl -X POST https://api.render.com/...
```

12.2 Frontend (Vercel)

```
{
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "env": {
    "VITE_API_URL": "@api_url"
  },
  "routes": [
    { "src": "^/api/.*", "status": 404 },
    { "handle": "filesystem" },
    { "src": "/(.*)", "dest": "/index.html" }
  ]
}
```

Environment Variables

```
VITE_API_URL=https://api.atlus.com
VITE_LOG_LEVEL=info
```

13. OPERAÇÃO E MANUTENÇÃO

13.1 Monitoramento

Métricas Críticas

- Response time (ms)
- Error rate (%)
- Database connection pool
- JWT validation failures
- Tenant isolation violations

Alertas

- Error rate > 5%
- Response time > 500ms
- Database down
- Disk space < 10%

13.2 Backups

```
# Backup diário automático (Render gerencia)
# Backup manual:
pg_dump -h db.render.com -U user atlas_db > backup_2026-08-22.sql

# Restore:
psql -h db.render.com -U user atlas_db < backup_2026-08-22.sql
```

13.3 Troubleshooting

User não consegue fazer login

- Validar RGA no SIA
- Validar hash de senha (bcrypt)
- Validar JWT secret

Error 403 ao acessar atletica

- Validar TenantGuard
- Validar JWT inclui directorias[]
- Validar diretoriaAtiva está em directorias

Desconto não aplicado

- Validar PriceCalculationService
- Validar role do usuário
- Validar desconto da diretoria

13.4 Operações Comuns

Adicionar nova atletica

1. Admin cria diretoria com codigo_curso
2. Aluno usa código para adicionar no perfil

3. Sistema valida código
4. Cria usuario_diretoria

Mudar desconto

1. Admin vai para Diretorias
2. Edita diretoria
3. Altera campo desconto
4. Próximas compras/inscrições usam novo desconto

Desativar atletica

1. Admin marca diretoria como ativo=false
2. Users não podem mais trocar para ela
3. TenantGuard rejeita acesso

APÊNDICE A: Glossário

Termo	Definição
Atlética	Organização estudantil em uma universidade
Tenant	Instância isolada de atletica
RGA	Registro Geral Acadêmico (matrícula do aluno)
JWT	JSON Web Token (token de autenticação stateless)
TenantGuard	Guard que valida isolamento de tenants
PermissionGuard	Guard que valida permissões por role
Multi-Atletica	Suporte a usuário ter acesso a múltiplas atleticas
Desconto Automático	Desconto aplicado automaticamente para sócios
código_curso	Identificador único de um curso (mapeia para atletica)

APÊNDICE B: Stack Resumido

Backend: NestJS + TypeScript + Prisma + PostgreSQL
Frontend: React 18 + Vite + Tailwind + Axios + React Query
Infrastructure: Render (backend) + Vercel (frontend) + Cloudflare (CDN)

Auth:	JWT + bcrypt
Monitoring:	Logs estruturados + Sentry
CI/CD:	GitHub Actions

Versão: 2.1

Status: Pronto para Produção

Data: 2026-08-22

Próxima Revisão: Pós-launch (feedback de usuários)